

DSAA Lab 04

Stacks

Areeba (TCT-004)

Thar Institute of Engineering Sciences and Technology (TIEST)

CT-159: Data Structure Algorithms and Applications (Pr)

Engr. Aneel Kumar

October 30, 2025

Exercise

- Q1: Please write a program which performs the following tasks:
- Make a left to right scan of the postfix expression
 - If the element is an operand push it on Stack
 - If the element is operator, evaluate it using as operands the correct number from stack and pushing the result onto the stack

Answer:

```
import java.util.Scanner;
import java.util.Stack;

public class postfixevaluation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter postfix expression (like 23*54*+9-): ");
        String exp = sc.nextLine();

        Stack<Integer> stack = new Stack<>();

        // Scan the expression from left to right
        for (int i = 0; i < exp.length(); i++) {
            char ch = exp.charAt(i);

            // If it's an operand (digit), push to stack
            if (Character.isDigit(ch)) {
                stack.push(ch - '0'); // convert char to int
            }
            // If it's an operator, pop operands and evaluate
            else {
                int val2 = stack.pop();
                int val1 = stack.pop();
                int result = 0;

                if (ch == '+') result = val1 + val2;
                else if (ch == '-') result = val1 - val2;
                else if (ch == '*') result = val1 * val2;
                else if (ch == '/') result = val1 / val2;

                // Push the result back to stack
                stack.push(result);
            }
        }

        // Final result will be on top of the stack
        System.out.println("Result: " + stack.pop());
    }
}
```

```

public class postfixevaluation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter postfix expression (like 23*54++9-): ");
        String exp = sc.nextLine();

        Stack<Integer> stack = new Stack<>();

        // Scan the expression from left to right
        for (int i = 0; i < exp.length(); i++) {
            char ch = exp.charAt(i);

```

Run postfixevaluation

```

C:\Users\Areeba\jdk\valhalla-ea-23-valhalla+1-90\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=56
Enter postfix expression (like 23*54++9-): 23*54++9-
Result: 17
Process finished with exit code 0

```

Q2: Write a java program to check a stack is full or not using an array with push and pop operations

Answer:

```

import java.util.Scanner;
class Stacks {
    int top;
    int size;
    int arr[];

    // Constructor
    Stacks(int size) {
        this.size = size;
        arr = new int[size];
        top = -1;
    }

    // Push operation
    void push(int value) {
        if (isFull()) {
            System.out.println("Stack is full!");
        } else {
            top++;
            arr[top] = value;
        }
    }

    // Pop operation
    void pop() {
        if (isEmpty()) {
            System.out.println("Stack is empty!");
        } else {
            top--;
        }
    }

    // Check if stack is full
    boolean isFull() {

```

```

        return top == size - 1;
    }

    // Check if stack is empty
    boolean isEmpty() {
        return top == -1;
    }

    // Display stack elements
    void display() {
        for (int i = 0; i <= top; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}

public class StackFullCheck {
    public static void main(String[] args) {
        // First example
        Stacks s1 = new Stacks(3);
        s1.push(1);
        s1.push(2);
        s1.push(3);
        System.out.println("Stack size: " + s1.size);
        System.out.print("Original Stack: ");
        s1.display();
        System.out.println("Push another value and check if the stack is full or not!");
        s1.push(4); // will show full message

        // Second example
        Stacks s2 = new Stacks(3);
        s2.push(10);
        s2.push(20);
        System.out.println("\nStack size: " + s2.size);
        System.out.print("Original Stack: ");
        s2.display();
        System.out.println("Check the said stack is full or not!");
        if (s2.isFull())
            System.out.println("Stack is full!");
        else
            System.out.println("Stack is not full!");
    }
}

```

The screenshot shows an IDE window with the file `StackFullCheck.java` open. The code is the same as shown in the previous block. The `Run` console at the bottom displays the following output:

```

Stack size: 3
Original Stack: 1 2 3
Push another value and check if the stack is full or not!
Stack is full!

Stack size: 3
Original Stack: 10 20
Check the said stack is full or not!
Stack is not full!

```

Q3: Write a java program to implement two stacks in a single array and performs push and pop operations for both stacks.

Answer:

```
class TwoStacks {
    int size;
    int top1, top2;
    int arr[];

    // Constructor
    TwoStacks(int n) {
        size = n;
        arr = new int[n];
        top1 = -1; // Stack 1 starts from left
        top2 = n; // Stack 2 starts from right
    }

    // Push in Stack 1
    void push1(int value) {
        if (top1 + 1 == top2) {
            System.out.println("Stack Overflow!");
        } else {
            top1++;
            arr[top1] = value;
        }
    }

    // Push in Stack 2
    void push2(int value) {
        if (top2 - 1 == top1) {
            System.out.println("Stack Overflow!");
        } else {
            top2--;
            arr[top2] = value;
        }
    }

    // Pop from Stack 1
    int pop1() {
        if (top1 == -1) {
            System.out.println("Stack 1 Underflow!");
            return -1;
        } else {
            int value = arr[top1];
            top1--;
            return value;
        }
    }

    // Pop from Stack 2
    int pop2() {
        if (top2 == size) {
            System.out.println("Stack 2 Underflow!");
            return -1;
        } else {
            int value = arr[top2];
            top2++;
        }
    }
}
```

```

        return value;
    }
}

// Display Stack 1
void display1() {
    for (int i = top1; i >= 0; i--) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

// Display Stack 2
void display2() {
    for (int i = top2; i < size; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}
}

public class TwoStacksInArray {
    public static void main(String[] args) {
        TwoStacks ts = new TwoStacks(10); // single array of size 10

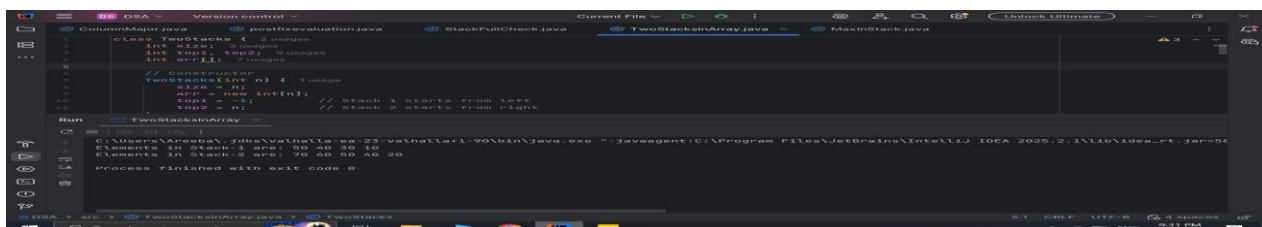
        // Push elements in Stack 1
        ts.push1(10);
        ts.push1(30);
        ts.push1(40);
        ts.push1(50);

        // Push elements in Stack 2
        ts.push2(20);
        ts.push2(40);
        ts.push2(50);
        ts.push2(60);
        ts.push2(70);

        System.out.print("Elements in Stack-1 are: ");
        ts.display1();

        System.out.print("Elements in Stack-2 are: ");
        ts.display2();
    }
}

```



Q4: Write a java program to find the maximum element in a stack.

Answer:

```
import java.util.Stack;

public class MaxInStack {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();

        // Push elements into the stack
        stack.push(5);
        stack.push(2);
        stack.push(1);
        stack.push(6);
        stack.push(8);

        System.out.println("Current stack elements:");
        display(stack);
        System.out.println("Maximum element: " + findMax(stack));

        // Remove two elements
        stack.pop();
        stack.pop();

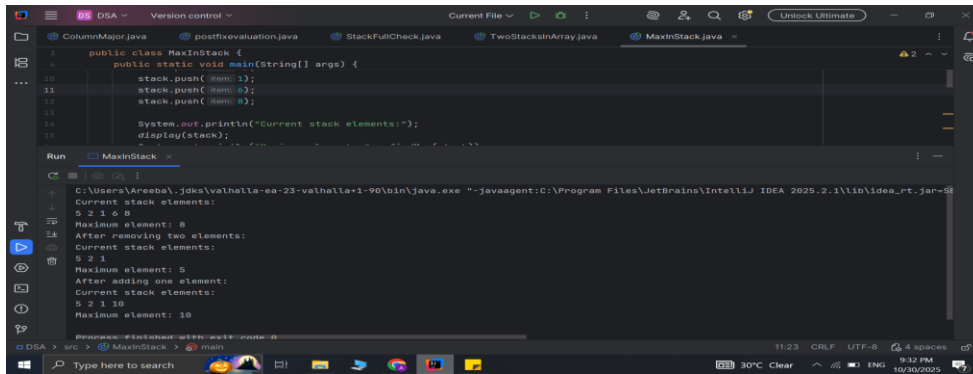
        System.out.println("After removing two elements:");
        System.out.println("Current stack elements:");
        display(stack);
        System.out.println("Maximum element: " + findMax(stack));

        // Add one element
        stack.push(10);

        System.out.println("After adding one element:");
        System.out.println("Current stack elements:");
        display(stack);
        System.out.println("Maximum element: " + findMax(stack));
    }

    // Method to find maximum element in stack
    static int findMax(Stack<Integer> s) {
        int max = s.get(0);
        for (int i = 1; i < s.size(); i++) {
            if (s.get(i) > max)
                max = s.get(i);
        }
        return max;
    }

    // Method to display stack elements
    static void display(Stack<Integer> s) {
        for (int i = 0; i < s.size(); i++) {
            System.out.print(s.get(i) + " ");
        }
        System.out.println();
    }
}
```



The screenshot shows an IDE window with a Java file named `MaxInStack.java`. The code defines a `MaxInStack` class with a `main` method that pushes elements 1, 6, and 8 onto a stack, prints the current elements, removes two elements, prints the current elements, adds one element, prints the current elements, and finally prints the maximum element (10).

```
public class MaxInStack {  
    public static void main(String[] args) {  
        stack.push(1);  
        stack.push(6);  
        stack.push(8);  
        System.out.println("Current stack elements:");  
        display(stack);  
        // ...  
    }  
}
```

The Run window shows the output of the program:

```
C:\Users\Areeba\jdk\valhalla-ea-23-valhalla-i-99\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.1\lib\idea_rt.jar=58  
Current stack elements:  
5 2 1 6 8  
Maximum element: 8  
After removing two elements:  
Current stack elements:  
5 2 1  
Maximum element: 5  
After adding one element:  
Current stack elements:  
5 2 1 10  
Maximum element: 10
```

The bottom status bar shows the file is at `src > MaxInStack > main`, the encoding is `UTF-8`, and the line length is `4 spaces`. The system tray shows the date and time as `10/30/2025 9:32 PM`.